

Building an Acehnese → Indonesian Translator

How I did it: data, training, testing and the app

Mutaib Rather · Universitas Syiah Kuala · Project report, 9 October 2026 (draft)

1. What I wanted to do

Acehnese has very little translated text, and general translation models know it only weakly. My goal was to make a model that translates Acehnese into Indonesian better than the base model does, to test it honestly on sentences it never saw in training, and to put it in an app that someone else can run on their own computer.

I did not have enough data to train a translation model from scratch, so I fine-tuned a pretrained model: Meta's NLLB-200 (600 million parameters), which already includes Acehnese (code `ace_Latn`).

2. The data

I used four sources. Each one has a different role, and I kept the test sentences completely separate from training.

Source	What it is	How I used it	Licence
NusaX-MT	1,000 human translations, Acehnese ↔ Indonesian (500 train, 100 validation, 400 test)	Training, validation and test, as split by the authors	CC-BY-SA 4.0
FLORES-200	Professional translations of Wikipedia-style sentences (about 1,000 dev and 1,012 devtest)	Dev minus 200 sentences for training, 200 for validation, devtest for testing	CC-BY-SA 4.0
Hikayat Abu Sammah (Depdikbud)	An old Acehnese verse story with its own Indonesian translation, in the same book	1,654 aligned verse pairs that passed screening: 1,432 train, 79 validation (pages 47–48), 143 test (pages 50–53)	Government book, free to distribute; reuse terms still being confirmed
AcehX (student corpus)	164,778 Acehnese lines, no Indonesian side	Only to detect duplicates and as a spelling reference. Never used as training pairs	Unknown, to confirm

I deliberately left out: song lyrics (copyrighted and garbled), a copyrighted Acehnese–Indonesian dictionary (usable only for personal lookup), and the other song and poem files from Andrie, which have no Indonesian side and whose authorship is unclear.

3. Turning the hikayat book into pairs

The book's PDF has a text layer, but it was unreliable: on a sample I checked by hand, 11.3% of words were wrong. So I re-read every page with OCR at 300 dpi (RapidOCR). On the same sample the error rate fell to 0.8%. Then I did the following steps:

- Removed page numbers, chapter headings, footnotes and lines the OCR was not confident about.
- Joined lines that the layout had split in two.

- Fixed common OCR letter mistakes (for example e read as c, or h as b), but only when the corrected word appears often in reference texts. I never changed words of four letters or fewer, because short words such as *bak* are real words.
- Aligned the Acehnese verses with the Indonesian verses. I used dynamic programming, which keeps the order of the text and matches shared words and similar word roots. Most verses are translated line for line, so one-to-one matches are preferred.
- Labelled every pair KEEP or REVIEW instead of deleting anything.

A read-through of 40 random KEEP pairs found all 40 to be correct translations of each other. 199 of the book's lines already appear in AcehX, so they are marked as duplicates. The book's screening results are in *reports/hikayat_abu_sammah.md*.

4. Keeping the test honest

If the model had seen the test sentences during training, its scores would be too high. I prevented that in three ways:

- The test and validation pages of the book are kept as whole blocks, so neighbouring verses do not end up on both sides.
- Any training pair that matches a validation or test sentence is removed. Matching ignores capital letters, diacritics, punctuation and old spellings.
- Any training pair whose Acehnese side is 90% or more similar to a test sentence is removed as well.
- The training script refuses to start if a leak remains. In the current build there are 0 leaks.

5. Preparing the text

Before training and before translating, Acehnese text goes through the same normalisation. It cleans up Unicode, converts Dutch-era spellings (*njang* → *nyang*, *djipeugot* → *jipeugot*) and removes the two-dot marks that Indonesian does not use (*löen* → *loen*, Andrie's rule). The original text is always kept next to the normalised version. The app uses exactly the same normalisation, and a test checks that the two match.

6. Training the model

Setting	Value	Why
Base model	NLLB-200 distilled 600M	Already knows Acehnese (ace_Latn)
What is trained	All weights	The model needs to adapt to this data
Optimiser	Adafactor, learning rate 1e-4, 10% warm-up	Uses little memory, so it fits a free Colab GPU
Batch size / length	8 pairs, up to 128 tokens	Fits on a T4 GPU
Label smoothing	0.1	Stops the model being overconfident on small data
Stopping	At most 10 epochs; stop after 2 epochs with no gain on validation	Prevents over-fitting
Model selection	Best epoch on validation chrF++	Test sets are never used to choose

Training ran on a free Google Colab T4 GPU. The notebook trains, saves the model and prints the results table. The exact training size is printed in the notebook output (about 1,900 pairs without FLORES, about 2,700 with it).

7. Testing

I score each translation with chrF++ (the main score), chrF and BLEU, using sacreBLEU. Higher is better. chrF++ compares characters and short word groups with a human translation, so it is a fair measure for a small model. Each test set is scored once at the end, for the base model and for mine. As a floor, I also score the trivial answer of copying the Acehnese text unchanged: on the NusaX test set that gives chrF 35.9 and BLEU 6.3.

Results. These are filled in from the training run (*models/ace-id-nllb/RESULTS.md*):

Test set	Sentences	Base NLLB-200 (chrF++)	My model (chrF++)	Change
NusaX test (modern text)	400	[fill in]	[fill in]	[fill in]
FLORES devtest (Wikipedia-style)	1,012	[fill in]	[fill in]	[fill in]
Hikayat test pages 50-53 (old verse)	143	[fill in]	[fill in]	[fill in]

The three test sets measure different things. The NusaX and FLORES sets are modern text, and the hikayat set is old verse, so a good score on one does not guarantee a good score on another. I report all three.

8. The app

- It runs on your own computer at <http://127.0.0.1:7860>. On a Mac you start it with *run_app.sh*; on Windows with *run_app.bat*.
- It works offline after setup. The first start downloads the model once; after that the model is read from the project folder.
- It normalises the input exactly as the training data was normalised, splits the text into sentences, and translates each sentence.
- It shows the result sentence by sentence and includes an accuracy panel with the test scores.
- It warns when the input is only one or two words, because short phrases are ambiguous.

9. Quality checks

- **Automated tests:** 69 tests, covering the leak check, normalisation, OCR, alignment, the training loop, and the app and launcher.
- **Language check:** on held-out NusaX sentences, a character-based classifier recognised Acehnese at 96–98% and Indonesian at 99.5–100%.
- **OCR check:** 0.8% word error on the hand-checked sample, against 11.3% for the PDF's own text layer.
- **Alignment check:** 40 of 40 random pairs were correct translations of each other.

10. Limits of this project

- **Small data.** About 1,900–2,700 training pairs is far below what translation models usually need.
- **Vocabulary.** About two-thirds of the running words in the general Acehnese text (AcehX) appear in the training data. About 13–14% of the words in the test sentences never appear in training. Expect weaker translations for unfamiliar words, names and dialect forms.
- **Different text types.** The hikayat is old literary Indonesian and NusaX is modern review text.

- **No native-speaker review yet** beyond the sample checks above.
- **Licences.** The hikayat book's reuse terms, AcehX's licence and the Quran translation still need confirmation. NLLB's weights are non-commercial (CC-BY-NC 4.0), so the model is for research and teaching use.

11. Next steps

- Have an Acehnese speaker check about 50 random pairs.
- Confirm the licences, then decide what can be published with the model and the pairs.
- Contribute new Acehnese text to AcehX if its maintainers allow it, keeping all test sentences out.
- Look for more bilingual material, because more pairs is the most reliable way to improve the model.

12. Tools and AI assistance

I used Claude, an AI assistant, to write most of the code: the data pipeline, the training script, the web app, the launcher and the tests. I directed the work and decided the data sources, the split rules and the evaluation. I ran the training and the tests myself, and I checked the results against the reports. The scores and examples in this report come from those runs, and all of the numbers can be re-created from the repository.

Note for the reader: check your course's rule on AI tools and adjust this section if the rule requires it.